

CONSISTENCY & COORDINATION

01

Quorum (R + W > N)

— read/write overlap; the knob for consistency vs latency

Read repair / anti-entropy

— heal stale replicas on read & in background

Merkle tree

— hash tree to find divergent replica ranges cheaply

Vector clock / version vector

— detect concurrent (conflicting) writes

Last-write-wins (LWW)

— timestamp resolution; simple but silently lossy

CRDT

— data type that merges concurrent writes without conflict

Lease

— time-bounded lock; underpins leader election & TTLs

Fencing token

— monotonic token blocks a stale leader from acting

Leader election

— pick one writer/coordinator — Raft, Paxos, ZAB

Distributed lock

— mutual exclusion across nodes (ZK, etcd, Redlock)

Clock skew

— node clocks disagree; breaks timestamp ordering

DESIGN PRINCIPLES

02

SOLID

— SRP, Open-Closed, Liskov, Interface-segregation, Dependency-inversion

DRY / KISS / YAGNI

— don't repeat · keep it simple · you aren't gonna need it

Separation of concerns

— isolate distinct responsibilities

Cohesion & coupling

— high cohesion within, loose coupling between modules

Dependency inversion

— depend on abstractions, not concretions

Law of Demeter

— talk only to immediate collaborators

Least astonishment

— behave the way users expect

Encapsulate what varies

— isolate change behind a stable interface

Idempotency

— same op, repeated, same result — safe to retry

Fail fast / fail safe

— surface errors early vs degrade gracefully

DOMAIN & MODELING

03

DDD

— Domain-Driven Design; model around the business domain

Bounded context

— explicit boundary where a model + language applies

Ubiquitous language

— shared vocabulary between devs and domain experts

Aggregate

— consistency boundary around a cluster of entities

Entity vs value object

— identity-bearing vs interchangeable-by-value

Anti-corruption layer

— translation shield between two models

Event storming

— workshop to discover domain events & flows

CQRS

— separate command (write) and query (read) models

Event sourcing

— store the event log; derive state by replay

Context map

— relationships between bounded contexts

DISTRIBUTED SYSTEMS

04

CAP / PACELC

— consistency–availability tradeoff under partition (+ latency else)

Eventual consistency

— replicas converge given no new writes

Linearizability

— strongest single-object consistency; acts like one copy

Consensus

— agreement among nodes — Paxos, Raft

Consistent hashing

— minimal reshuffle when nodes change

Sharding / partitioning

— split data by key across nodes

Hot partition / key skew

— one shard overloaded by a bad partition key

Replication lag

— delay before a replica reflects a write

Saga

— distributed txn via local steps + compensations

Backpressure

— consumer signals producer to slow down

Fan-out: write vs read

— precompute feeds on post vs assemble on read

Thundering herd / stampede

— mass simultaneous hits; fix w/ coalescing + jitter

At-least-once + dedup

— real-world default; consumers must be idempotent

Exactly-once (effectively)

— at-least-once + idempotency; true E1 impossible across failures

Split-brain

— partitioned nodes both think they're leader (→ fencing)

CLOUD-NATIVE & PLATFORM

05

Container / Kubernetes

— packaged runtime + orchestrator for them

Service mesh

— infra layer for service-to-service comms (Istio, Linkerd)

Sidecar

— helper container alongside the main one

Serverless / FaaS

— run code without managing servers; scale-to-zero

IaC

— Infrastructure as Code (Terraform, Pulumi)

GitOps

— Git as source of truth for declarative infra

Platform engineering^{NEW}

— building an internal platform as a product

IDP^{NEW}

— Internal Developer Platform; self-service golden paths

WebAssembly (Wasm)^{NEW}

— sandboxed, sub-ms-startup polyglot runtime, now server-side

eBPF^{NEW}

— safe kernel programs powering networking/observability/security

Circuit breaker — stop calling a failing dependency; closed/open/half-open

Bulkhead — isolate resource pools so one failure can't sink all

Retry + backoff + jitter — tolerate transients without a retry storm

Rate limiting / throttling — cap request rate (token bucket, sliding window)

Load shedding — reject excess work to protect the rest

Graceful degradation — reduced function instead of full outage

Autoscaling — add/remove capacity to track demand

Blast radius — scope of impact when something fails

Cell-based / shuffle sharding — isolate users into independent cells

Chaos engineering — inject failure on purpose to prove resilience

Metastable failure — system stays down after the trigger clears

DATA ARCHITECTURE

OLTP vs OLAP — transactional vs analytical workloads

LSM-tree vs B-tree — write-optimized (append+compact) vs read-optimized

WAL — write-ahead log; durability + basis for replication/CDC

Bloom filter — probabilistic "definitely-not-present" to skip disk reads

Write/read amplification — hidden cost of indexes & LSM compaction

ETL / ELT — transform-then-load vs load-then-transform

Data lake / warehouse — raw object store vs structured analytic store

Data lakehouse ^{NEW} — lake openness + warehouse reliability in one

Open table formats ^{NEW} — Iceberg / Delta / Hudi / Paimon — ACID tables on files

Data mesh ^{NEW} — decentralized, domain-owned data products

Data contract ^{NEW} — enforced schema/quality pact between producer & consumer

CDC — Change Data Capture; stream every DB change off the log

RAG — Retrieval-Augmented Generation; ground an LLM in external knowledge

Vector database — store embeddings for similarity (ANN) search

Embeddings — vector representation of meaning

Semantic / hybrid search — meaning-based, optionally fused with keyword

Chunking — splitting docs for indexing/retrieval

Re-ranking — reorder retrieved candidates by relevance

Agent / agentic — LLM that plans & calls tools in a loop

MCP — Model Context Protocol; standard tool/data connector layer

LLM gateway — proxy for routing, caching, cost & rate control of model calls

Guardrails — input/output safety & policy filters

Grounding / hallucination — anchoring to facts vs confident fabrication

Eval — systematic quality measurement of model outputs

Fine-tune vs prompt vs RAG — retrain vs instruct vs retrieve — three knowledge levers

DELIVERY & OBSERVABILITY

CI/CD — continuous integration / delivery pipelines

Blue-green / canary — swap environments vs gradual rollout

Feature flags — decouple deploy from release; toggle at runtime

DORA metrics — deploy freq, lead time, MTTR, change-fail rate

SLI / SLO / error budget — measured signal, target, allowable failure

Observability (3 pillars) — logs · metrics · traces

OpenTelemetry — vendor-neutral telemetry standard

FinOps ^{NEW} — cloud cost accountability; unit economics & chargeback

INTEGRATION & APIS

REST / GraphQL / gRPC — resource · query · high-perf RPC styles

Event-driven architecture — components react to events asynchronously

Pub-sub / message broker — decoupled fan-out via topics (Kafka, Pub/Sub)

API gateway / BFF — single entry point / per-client tailored API

Webhook — server-to-client HTTP callback on events

Idempotency key — client token making a retried call safe

Schema registry — governs event/message schema evolution

Contract testing — verify producer/consumer agreement (Pact)

Outbox pattern — atomic DB write + reliable event publish